

Compléments d'Algorithmique

Chapitre 8 : Paradigmes et langages

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 8. Paradigmes et langages

1 Paradigmes de programmation

2 Langages de programmation

3 Types de données

Paradigmes de programmation

Paradigmes de programmation

Vous êtes déjà familiers avec certains paradigmes de programmation :

- la **programmation impérative** (au Q1, en Python),
- la **programmation orientée objet** (au Q2, en Python et en Java),
- et, sans que cela ait été dit explicitement, vous avez fait un peu de **programmation fonctionnelle** (voir ci-après).

Certains langages sont plus orientés vers un paradigme que d'autres. Par exemple, Python est un langage multi-paradigme qui supporte la programmation impérative, la programmation orientée objet et la programmation fonctionnelle. Java est principalement orienté objet, mais il supporte aussi la programmation fonctionnelle (depuis Java 8).

Nous allons brièvement passer en revue les paradigmes de programmation les plus courants et les langages qui leur sont associés. Vous en approfondirez certains dans le cadre de vos études.

Programmation Impérative

- **Principe** : Programmation basée sur des procédures et des instructions qui décrivent les étapes à suivre.
- **Avantages** : Facile à comprendre et à apprendre, généraliste.
- **Langages typiques** : Fortran, Pascal, Basic, Ada, C, Python.

L'impératif est le paradigme dominant dans la plupart des langages généralistes, même si d'autres paradigmes (POO, fonctionnel) sont souvent intégrés dans les langages plus modernes.

Programmation Impérative

Exemple en C

```
#include <stdio.h>

int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main() {
    int n = 5;
    printf("Factorielle de %d = %d\n", n, factorial(n));
    return 0;
}
```

Programmation Orientée Objet

- **Principe** : Organisation du code autour d'objets et de classes, encapsulation des données, polymorphisme et héritage.
- **Avantages** : Réutilisabilité, modularité.
- **Langages typiques** : Java, C++, Python, Ruby.

Programmation Orientée Objet

Exemple en C++

```
class Person {  
    private:  
        std::string name;  
  
    public:  
        Person(std::string n) {  
            name = n;  
        }  
        void greet() {  
            std::cout << "Bonjour, je suis " << name << " !" << std::endl;  
        }  
};  
  
int main() {  
    Person p1("Alice");  
    p1.greet();  
    return 0;  
}
```


Programmation Fonctionnelle

- **Principe** : Encourager l'utilisation de fonctions pures (sans effet de bord), l'immuabilité des données, et la composition des fonctions.
- **Avantages** : Peu d'effets de bord, code plus prévisible, testable, et facilement parallélisable.
- **Langages typiques** : Haskell, Lisp, Scala, OCaml, Python.

Les fonctions lambda en Python sont inspirées de la programmation fonctionnelle : elles sont sans état (pas de variables modifiables internes) souvent utilisées pour la composition de fonctions, et peuvent être passées comme arguments à d'autres fonctions.

Programmation Fonctionnelle

Exemple en Haskell

```
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n - 1)

main :: IO ()
main = do
    let n = 5
    print (factorial n)
```

Programmation Logique

- **Principe** : Basée sur la déclaration de faits et de règles, le programme trouve les solutions en utilisant un algorithme d'inférence (inspiré de preuves logiques).
- **Avantages** : expressivité (“déclarer” un problème par des faits et des règles plutôt que de concevoir une solution impérative), beaucoup utilisée en IA et pour des problèmes de satisfaction de contraintes.
- **Langages typiques** : Prolog, Datalog, ASP, CLP.

Programmation Logique

Exemple en Prolog

```
parent(edward, ethel).  
parent(ethel, alan).
```

```
ancetre(X, Y) :- parent(X, Y).  
ancetre(X, Y) :- parent(X, Z), ancetre(Z, Y).
```

Résultat de la requête `ancetre(X, alan).`

```
X = ethel ;  
X = edward ;  
false.
```

Grâce aux deux faits (`parent`) et aux deux règles (`ancetre`), Prolog a pu inférer (une partie) de l'arbre généalogique d'Alan Turing.

Programmation Événementielle

- **Principe** : Basée sur la gestion des événements (clics, entrées clavier, etc.).
- **Avantages** : Essentiel pour les interfaces graphiques et le développement web.
- **Langages typiques** : JavaScript, C#, Python (Tkinter).

Exemple en JavaScript

```
document.getElementById("btn").addEventListener("click", function() {  
    alert("Bouton cliqué !");  
});
```

Nous en avons fait au Chapitre 6 (GUI) avec Tkinter.

Programmation Concurrente

- **Principe** : Basée sur l'exécution parallèle de plusieurs tâches.
- **Avantages** : Optimise les performances pour les systèmes multi-cœurs.
- **Langages typiques** : Java, Go, Python (multiprocessing).

Programmation Concurrente

Exemple en Python

```
import concurrent.futures
import time

def worker(n):
    print(f"Start task {n}")
    time.sleep(2) # Simule une tâche longue
    print(f"End task {n}")
    return n * n

if __name__ == "__main__":
    with concurrent.futures.ProcessPoolExecutor(max_workers=3) as exec:
        results = exec.map(worker, range(5))
        # Exécute worker(0), worker(1), ... en parallèle

    print("Results:", list(results))
```

(Exécution en séance)

Langages de programmation

Langages de programmation

Il existe quantité de langages de programmation, certains sont spécifiques à un domaine (ex. SQL pour les bases de données), d'autres sont généralistes (ex. Python, Java).

- **Langages compilés** : C, C++, Rust, Go, etc.
- **Langages interprétés** : Python, Ruby, JavaScript, etc.
- **Langages mixtes** : Java (compilé en bytecode), C# (compilé en CLR).
- **Langages de balisages et description de documents** : LaTeX, Markdown, HTML, XML.
- **Langages de script** : Bash, Perl, Python, etc.
- **Langages de requête** : SQL, SPARQL, etc.
- ...

Historique visuel des langages

Vidéo de *Data is Beautiful* sur Youtube (octobre 2019) qui montre la popularité des principaux langages de 1965 à 2019 :

<https://www.youtube.com/watch?v=Og847HVwRSI>

Classement des langages en 2024

- 1 Python : simplicité et polyvalence ;
- 2 JavaScript : développement web (dont lib. React, très populaire) ;
- 3 TypeScript : croissance de 37,8% en 2024
- 4 Java : utilisé par 44,8% des développeurs
- 5 C# : principalement développement Windows
- 6 C++ : performances système
- 7 PHP : développement web
- 8 Shell : automatisation
- 9 C : développement système
- 10 Go : cloud computing

Source : <https://www.wildcodeschool.com> (6 novembre 2024)

Typage

Le **typage** en programmation désigne la manière dont le langage gère les types de données (entiers, flottants, chaînes, etc.).

Les principales caractéristiques du typage sont :

- **explicite** ou **implicite** : il est explicite si le programmeur doit déclarer le type des variables ; et implicite si le type est inféré automatiquement ;
- **statique** ou **dynamique** : il est statique si le type des variables est défini à la compilation et ne peut pas changer ; et dynamique s'il déterminé à l'exécution ;
- **fort** ou **faible** : il est fort si les opérations entre types incompatibles sont interdites ; et faible si des conversions automatiques sont possibles.

Typage

Exemple de typage explicite (C)

```
int a = 42;  
float b = 3.14;
```

Exemple de typage implicite (Python)

```
a = 42  
b = 3.14
```

Le typage explicite est souvent considéré comme plus clair, le typage implicite comme plus concis.

Typage

Exemple de typage statique (C)

```
int a = 42;  
a = 'hello'; // Erreur à la compilation
```

Exemple de typage dynamique (Python)

```
a = 42  
a = 'hello' # Devient un str à l'exécution
```

Le typage statique est souvent considéré comme plus sûr (moins d'erreurs à l'exécution), le typage dynamique comme plus flexible.

Typage

Exemple de typage fort (Python)

```
a = 42  
b = 'hello'  
c = a + b # Erreur à l'exécution
```

Exemple de typage faible (Javascript)

```
let x = "42" + 5; // "425"  
let y = "42" - 5; // 37
```

Le typage fort est souvent considéré comme plus sûr et clair, le typage faible comme plus flexible.

Observez comme le dernier exemple peut être surprenant. C'est une des raisons de l'apparition de TypeScript, qui est un sur-ensemble de JavaScript avec typage statique et fort.

Typage

Langage	Typage explicite	Typage statique	Typage fort
C/C++	Oui	Oui	Oui
Java	Oui	Oui	Oui
Python	Non	Non	Oui
JavaScript	Non	Non	Non
TypeScript	Oui	Oui	Oui

Annotations de types et Pyright

Depuis Python 3.6, il est possible d'annoter les types des variables et des fonctions. Cela permet de rendre le code plus clair et de faciliter la détection d'erreurs, mais cela ne change pas le comportement du code.

Pyright est un vérificateur de types statique pour Python qui utilise les annotations et permet de détecter des erreurs de type avant l'exécution. Il peut être inclus directement dans **VS Code** (via l'extension **Pylance**).

Exemple d'annotations de types

```
def add(a: int, b: int) -> int:
    return a + b
print(add(1, 2))      # OK
print(add(1, "2"))    # Erreur de type
```

Types de données

Types de données

La plupart des langages partagent les mêmes types de données de base (booléens, entiers, réels, chaîne, ...), mais la manière dont ils sont implémentés et utilisés peut varier.

Egalement, certains langages (typiquement plus anciens) imposent d'allouer et de désallouer manuellement (ou via des bibliothèques spécifiques) la mémoire pour les types de données (ex. C, C++), alors que d'autres langages gèrent la mémoire automatiquement via par ex. un **garbage collector** (ex. Python, Java) ou un système d'**ownership** (ex. Rust).

Types de données

Lié à la gestion de la mémoire, il y a aussi la notion de **pointeur** (ex. C, C++) qui est un type de données qui stocke l'adresse d'une autre variable. Cela permet de manipuler directement la mémoire, mais cela peut aussi être source d'erreurs si mal utilisé.

Nous n'allons pas rentrer dans les détails ici, mais il faut s'en informer quand on utilise un langage particulier.

Types de données

Il y a un point important à connaître quand on a commencé à apprendre la programmation avec Python : la notion de **tableaux**.

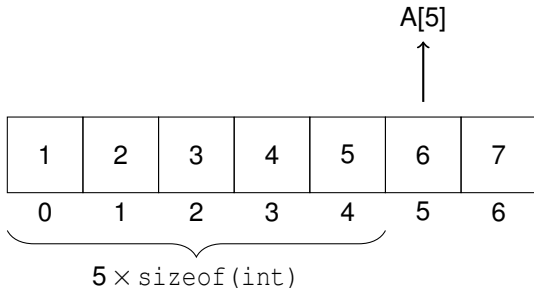
Un tableau est une structure de données qui permet de stocker plusieurs valeurs du même type (**homogène**), en spécifiant la taille de ce tableau à sa création.

Exemple de tableau en C

```
int tableau[7];  
tableau[0] = 1;  
tableau[1] = 2;  
...  
tableau[6] = 7;
```

Types de données

Le fait de déclarer un tableau de taille fixe n permet d'allouer de la mémoire contigüe pour n données d'un certain type. En C, la taille en mémoire d'une donnée d'un certain type est obtenu via `sizeof`. Cela permet d'accéder en $O(1)$ à un élément du tableau via son indice.



Exemple : L'emplacement en mémoire de l'élément $A[5]$ est décalé de $5 \times \text{sizeof}(\text{int})$ par rapport à l'adresse en mémoire du tableau A .

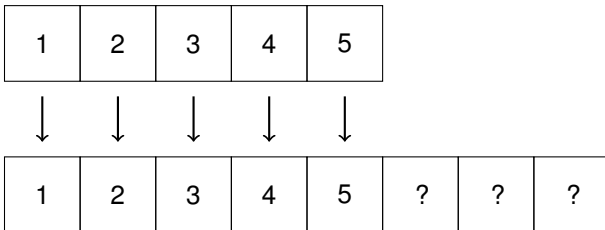
Types de données

Cette notion existe en Java (`array`) mais pas en Python, où les listes (`list`) sont dynamiques et peuvent contenir des éléments de types différents.

Cependant, en interne, les listes Python sont implémentées via des tableaux (ce qui permet l'accès en $O(1)$ via un indice). Ces tableaux sont dits **dynamiques** car ils peuvent changer de taille au cours de l'exécution du programme (voir ci-après).

Types de données

Si la taille du tableau sous-jacent d'une liste Python n'est plus assez grande, un nouveau tableau plus grand est créé et les premières données y sont copiées.



Le côté dynamique des listes Python peut donc entraîner des performances moins bonnes que les tableaux statiques en C ou Java, à cause de ce travail qui n'est pas en $O(1)$.

Types de données

Réallocation de mémoire en Python

```
import sys

t = []
print("Taille mémoire :", sys.getsizeof(t), 'octets')

for i in range(10):
    t.append(i)
    print(f"Ajout de {i}, taille mémoire : {sys.getsizeof(t)} octets")

print("Remplacement d'un entier par une longue chaîne")
t[0] = "x" * 1000000
print("Taille mémoire :", sys.getsizeof(t), 'octets')
```

Types de données

Résultat

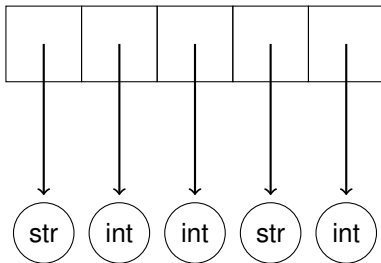
```
Taille mémoire : 56 octets  
Ajout de 0, taille mémoire : 88 octets  
Ajout de 1, taille mémoire : 88 octets  
Ajout de 2, taille mémoire : 88 octets  
Ajout de 3, taille mémoire : 88 octets  
Ajout de 4, taille mémoire : 120 octets  
Ajout de 5, taille mémoire : 120 octets  
Ajout de 6, taille mémoire : 120 octets  
Ajout de 7, taille mémoire : 120 octets  
Ajout de 8, taille mémoire : 184 octets  
Ajout de 9, taille mémoire : 184 octets  
Remplacement d'un entier par une longue chaîne  
Taille mémoire : 184 octets
```

On constate que le tableau a été redimensionné plusieurs fois pour s'adapter à la taille de la liste.

Par contre, le fait de remplacer un entier par une longue chaîne n'a pas entraîné de redimensionnement de la liste : [à votre avis, pourquoi ?](#)

Types de données

Réponse : car Python stocke les **références** vers les objets plutôt que les objets eux-mêmes.



C'est donc en réalité les adresses des objets qui sont stockées dans la liste, et non les objets eux-mêmes.

Types de données

Réallocation de mémoire en Python

```
import sys

def taille_totale(t):
    res = 0
    for i in t: # les objets référencés
        res += sys.getsizeof(i)
    res += sys.getsizeof(t) # les références
    return res

t = [1, 2, 3, 4, 5]
print("Taille totale :", taille_totale(t), 'octets')

print("Remplacement d'un entier par une longue chaîne")
t[0] = "x" * 1000000
print("Taille totale :", taille_totale(t), 'octets')
```

Types de données

Résultat

Taille totale : 244 octets

Remplacement d'un entier par une longue chaîne

Taille totale : 1000257 octets

Types de données

Les tableaux sous-jacents sont donc homogènes (tous les éléments sont des références), mais les objets référencés peuvent être de types différents.

C'est grâce à cette implémentation via un tableau de références (dont chacune est de taille fixe en mémoire) que Python permet l'accès en $O(1)$ à un élément de la liste via son indice.

Mais cela peut aussi entraîner des comportements inattendus comme on l'a déjà constaté (par exemple avec des listes de listes), quand on modifie un objet référencé.

So Long, and Thanks for All the Fish

Merci pour votre attention et beaucoup de succès pour la suite de vos études !